Stabilizing Training by Capping Local Representation Displacement

An intervention motivated by destructive-regime tracing

Analysis 065 / H048 / Revision 001 8 November 2026

Findings at a glance

We cap the local preactivation change caused by each proposed weight/bias update, holding current inputs fixed. All layers remain trainable. Distributed caps reduce mean training loss by **22.85%, 16.22%, and 26.00%** for MLP/CIFAR-10, ResNet/CIFAR-10, and Transformer/WikiText-2 in the selected windows; two further MLP initializations improve by 19.43% and 32.08%. Loss excursions shrink despite persistent or more frequent representation reversals. **Terminal test loss worsens under the distributed cap in every setup.** These finite-window optimization results give no generalization or convergence guarantee.

1. Motivation and diagnostic notation

We follow the manuscript: update t starts at θ_{t-1}, uses the raw full-objective gradient $g_t = \nabla\mathcal{L}(\theta_{t-1})$, and ends at θ_t. The raw-gradient diagnostics are

$$\rho_t = \cos(g_t, g_{t-1}), \qquad \eta\widehat{\kappa}_t = 1 - \rho_t \frac{\|g_t\|_2}{\|g_{t-1}\|_2} = 1 - \frac{\langle g_t, g_{t-1}\rangle}{\|g_{t-1}\|_2^2}. \tag{1}$$

Under Adam, the latter measures normalized projected gradient change, not a Hessian eigenvalue or stability threshold. A zero denominator is undefined.

Fixed data, changing representations. A probe is a chosen set of input examples kept identical across evaluated network states: same ordering, token contexts and normalization groups. A_t^ℓ contains their representations after update t. Rows index matching examples or token/spatial positions; columns index features. Data inputs stay fixed; hidden representations can change as the parameters change. With P_ℓ total scalar entries, define

$$\delta_t^\ell = \mathrm{vec}(A_t^\ell - A_{t-1}^\ell), \qquad \rho_t^\ell = \frac{\langle \delta_t^\ell, \delta_{t-1}^\ell\rangle}{\|\delta_t^\ell\|_2 \, \|\delta_{t-1}^\ell\|_2}, \qquad r_t^\ell = \frac{\|\delta_t^\ell\|_2}{\sqrt{P_\ell}}. \tag{2}$$

vec flattens without averaging: ten rows by five columns give a 50-dimensional vector. ρ_t^ℓ is one cosine over all matching sample/feature coordinates, not a mean of per-sample cosines. r_t^ℓ is their root mean square change. Raw-gradient alignment remains ρ_t. Negative cosine describes adjacent cancellation (*destructive* geometry), not necessarily rising loss.

What the earlier tracing established. In the MLP, reversal is already visible at the first learned projection and strongly preserved through both hidden layers; tanh attenuates movement but does not stop reversal. In ResNet, first-convolution reversal is also present, but similarity to the output's reversal timing strengthens in later residual stages: alignment-series correlation is 0.578 at the first convolution and 0.991 at the final stage. This is not monotonic growth in displacement with depth. In the Transformer, reversal is visible at the first attention output while embeddings can still move forward; this localizes a measured boundary, not an internal attention mechanism or a proven causal origin.

Temporal SVD concentrates output-change energy into relatively few patterns: 5, 16 and 15 modes explain 95% for MLP, ResNet and Transformer, respectively, over outgoing transitions from completed states 450–500, 280–310 and 552–600 (51, 31 and 49 updates). These patterns span samples and features, not individual neurons; rank is limited by the window length. The window-fitted basis is fixed, although instantaneous directions can rotate within it. This motivates amplitude control without establishing a universal rank or initiating layer. The cap uses neither cosine triggers nor SVD vectors; loss excursions and learning progress assess its effect.

2. Adam direction versus parameter increment

Adam's first and second moments are m_t and v_t, respectively:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t, & v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^{\odot 2}, \\ \widehat{m}_t &= m_t/(1-\beta_1^t), & \widehat{v}_t &= v_t/(1-\beta_2^t), \\ d_t &= \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t}+\epsilon}, & \Delta\theta_t^{\mathrm{Adam}} &= -\eta d_t. \end{aligned} \tag{3}$$

Operations are coordinatewise. Moments start at zero only at initialization; the intervention retains their values and global update count. The signed increment $\Delta\theta_t^{\mathrm{Adam}}$ contains the proposed weight and bias changes $\Delta W_t^{\mathrm{Adam}}$ and $\Delta b_t^{\mathrm{Adam}}$ for each module. Without intervention, $\theta_t = \theta_{t-1} - \eta d_t$. With the cap, $\theta_t = \theta_{t-1} + \Delta\theta_t^{\mathrm{cap}}$ instead. The cap changes the increment, not the current moments or Adam direction. Implementation scores use the realized increment after storing the native proposal in FP32; Eq. (3) gives its exact-arithmetic value.

3. Local displacement: dimensions and computation

Affine means a weighted sum plus bias. For one neuron, $z = w^\top h + b$ is its preactivation, before applying an activation such as tanh. **Direct affine response** means the change caused by its proposed weight and bias updates *with the same input* h: $\Delta z_{\mathrm{direct}} = (\Delta w)^\top h + \Delta b$. For $h = 2$, $\Delta w = 0.1$, $\Delta b = 0$, this change is 0.2. Earlier-layer updates can also change h; that incoming movement and subsequent nonlinear transformations are outside this direct contribution.

For one module (suppressing ℓ), let n count probe rows, d input features and m output units. Before update t,

$$\underbrace{H_{t-1}}_{n\times d}, \quad \underbrace{W_{t-1},\ \Delta W_t^{\mathrm{Adam}}}_{m\times d}, \quad \underbrace{b_{t-1},\ \Delta b_t^{\mathrm{Adam}}}_{m\times 1}, \quad \underbrace{\mathbf{1}_n}_{n\times 1}$$

are the module inputs, weights and proposed weight increments, biases and proposed bias increments, and the all-ones vector. The proposed local preactivation change, with these module inputs held fixed, is

$$\Delta Z_t^{\mathrm{local}} = H_{t-1}(\Delta W_t^{\mathrm{Adam}})^\top + \mathbf{1}_n(\Delta b_t^{\mathrm{Adam}})^\top \in \mathbb{R}^{n\times m}, \qquad s_{t,j} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\Delta Z_{t,ij}^{\mathrm{local}})^2}. \tag{4}$$

The $n \times m$ matrix has one row per example/position and one column per output unit, before the nonlinearity. Its entry (i, j) is the change for example/position i at unit j. Each $s_{t,j}$ averages squared changes over the n rows, then takes the square root: it gives **one score per output unit**, so the score vector has m entries. Signs cannot cancel; individual examples can exceed this RMS. For MLP linear 1, $(n, d, m) = (5000, 3072, 200)$: H is 5000×3072, ΔW is 200×3072, Δb has 200 entries, $\Delta Z^{\mathrm{local}}$ is 5000×200, and there are 200 scores.

How it is computed. The implementation makes one additional forward pass over the probe per update, saving the current inputs of targeted modules. After Adam proposes parameter increments, Eq. (4) needs only local matrix products (convolutions for convolutional modules), not a second full-network forward. Two full forwards before/after an update would instead include changes arriving from earlier layers. The next state's probe capture also supplies actual representation-change diagnostics. Extra input storage, FP64 products and a second local product checking FP32 rounding still cost time; the overhead is not universally twice training time and depends on probe size and targeted modules.

4. Calibration and selective update scaling

Let t_0 be the first update to which the cap is applied. For module ℓ with m_ℓ output units, collect its proposed scores from the preceding eight untreated Adam updates. Pool all $8m_\ell$ values and set

$$\tau_\ell = Q_q\Big(\{s^\ell_{k,j} : k = t_0 - 8, \dots, t_0 - 1,\ j = 1, \dots, m_\ell\}\Big), \tag{5}$$

where Q_q is the empirical quantile with linear interpolation. Thus $\tau_\ell \in \mathbb{R}_{\geq 0}$ is **one scalar per module**, shared by all its output units, with the units of preactivation change. It is fixed thereafter; the scores and scaling factors are recomputed every update. For MLP linear 1, calibration pools $8 \times 200 = 1600$ scores; $t_0 = 449$ uses updates 441–448. q50 uses the median ($q = 0.50$), q75 the 75th percentile ($q = 0.75$).

Intuition. Dampen a proposed update when its joint weight/bias change would move one unit's preactivation too much on the probe. This evaluates the proposed movement for current inputs, not an individual weight's size or a Jacobian sensitivity. For $\tau_\ell > 0$ and $s^\ell_{t,j} > 0$,

$$a^\ell_{t,j} = \min(1, \tau_\ell / s^\ell_{t,j}), \qquad \Delta W^{\ell,\mathrm{cap}}_{t,j:} = a^\ell_{t,j} \Delta W^{\ell,\mathrm{Adam}}_{t,j:}, \qquad \Delta b^{\ell,\mathrm{cap}}_{t,j} = a^\ell_{t,j} \Delta b^{\ell,\mathrm{Adam}}_{t,j}. \tag{6}$$

The vector a^ℓ_t has m_ℓ dimensionless entries. If a unit's score is twice its module's threshold, its entire weight-update row and bias increment are halved; below the threshold, they are unchanged. With $W \in \mathbb{R}^{m \times d}$, **output column j corresponds to weight row j**: all d weights feeding that unit receive the same multiplier, along with its bias. The resulting local RMS is $a^\ell_{t,j} s^\ell_{t,j} \leq \tau_\ell$ in exact arithmetic. Use $a = 1$ for zero score; a zero calibrated threshold disables the module's cap. Untargeted increments stay native. q50 does not require half the units to be modified at later updates; final storage rounding is checked separately.

Algorithm 1 Local representation displacement cap at update t

Require: Parameters θ_{t-1}, Adam state, fixed probe, target modules $\mathcal{S}$, budgets τ_ℓ

1: Retain θ_{t-1}; capture H^ℓ_{t-1} in one probe forward pass.
2: Compute the raw gradient g_t of the complete training objective.
3: Advance m_t, v_t and the clock; form d_t and the stored native proposal θ^+_t.
4: Set $\Delta\theta^{\mathrm{Adam}}_t = \theta^+_t - \theta_{t-1}$; initialize $\Delta\theta^{\mathrm{cap}}_t = \Delta\theta^{\mathrm{Adam}}_t$.
5: **for** each targeted module $\ell \in \mathcal{S}$ with $\tau_\ell > 0$ **do**
6: Extract $(\Delta W^{\ell,\mathrm{Adam}}_t, \Delta b^{\ell,\mathrm{Adam}}_t)$ from $\Delta\theta^{\mathrm{Adam}}_t$.
7: Use local affine products to compute one RMS score per output unit, Eq. (4).
8: Scale each proposed weight row and bias using Eq. (6).
9: **end for**
10: Store $\theta_t = \theta_{t-1} + \Delta\theta^{\mathrm{cap}}_t$; keep the advanced Adam state.
11: Measure loss and actual consecutive representation changes for evaluation.

Entry versus distributed. The entry variant targets MLP linear 1, ResNet conv 1, or the first Transformer QKV projection. The distributed variant targets both MLP hidden affines, all 31 ResNet convolutions, or both QKV projections plus all four feed-forward affines. MLP output weights, ResNet normalization/classifier parameters, and Transformer attention output projections, LayerNorm, embeddings and decoder are outside the cap. A convolutional score averages over images and spatial positions: its conceptual H rows are receptive-field patches, evaluated by convolution without materializing that matrix. QKV scores measure the projection, not the complete attention operation. Selection is over existing units/channels, not temporal singular vectors.

5. Experimental design and measurement timing

Case	Full objective per update	Fixed probe	LR; first capped update; final state
MLP / CIFAR-10	5,000 images; half squared output error	All 5,000 images, native 1,000-image chunks	.001; 449; 800/1152
ResNet / CIFAR-10	50,000 images; mean cross-entropy	First intact 1,000-image BatchNorm group	.003; 265; 576
Transformer / WikiText-2	2,088,600 valid tokens; mean NLL	First 64 length-35 sequences: 2,240 valid positions	.003; 553; 928

Table 1: Full-objective deterministic updates throughout. Probe coverage changes only measurement and the local constraint, not the training objective.

The MLP is $3072 \to 200 \to 200 \to 10$ with two tanh activations and a linear output. ResNet-32 has stage widths 16/32/64 and BatchNorm without running statistics. The two-block Transformer has width 200, two attention heads, feed-forward width 200, zero dropout and untied embeddings/decoder (vocabulary 33,278). All use Adam $(\beta_1, \beta_2) = (.9, .999)$, $\epsilon = 10^{-8}$, no weight decay or clipping, FP32 parameters and disabled TF32. Dataset order, cached data, initialization and source/config hashes are pinned. All gradients and moments continue to update; future gradients naturally depend on the intervened trajectory. For the MLP, $\mathcal{L} = (2N)^{-1} \|F - Y\|_F^2$ with one-hot targets Y. Output diagnostics retain raw linear outputs for MLP and all class/vocabulary log-probabilities for ResNet/Transformer, without averaging probe rows.

Figure indexing. The horizontal axis retains the recorded number of completed updates s. At s, loss is $\mathcal{L}(\theta_s)$, the next gradient is g_{s+1}, and outgoing motion is $\delta_{s+1}^{\ell} = \mathrm{vec}(A_{s+1}^{\ell} - A_s^{\ell})$. Hence the plotted cosines are ρ_{s+1} and ρ_{s+1}^{ℓ}; the gradient cosine concerns the preceding transition, while the representation cosine includes the next one. Labels now show this distinction; no recorded state or value is shifted. The first intervened update is $t = t_0$ (e.g., 449 after MLP state 448).

6. Results: cap variants and unchanged-Adam controls

The primary entry screen uses q75; the common distributed comparison uses q50. Each branch is paired with unchanged Adam from identical parameters and optimizer state at intervention start, on the same worker. Mean loss averages the stated loss states; tail averages the last 32 states; $\mathrm{PV} = \sum_s \max(\mathcal{L}(\theta_{s+1}) - \mathcal{L}(\theta_s), 0)$ uses adjacent states within that window. Outgoing motion also uses a retained state after its end. Maximum includes the shared initial state. Percent changes below are paired descriptive effects; no confidence intervals are claimed.

Model	Cap	States	Mean	Tail	Maximum	PV
MLP	Entry q75	448–800	−8.26%	+8.50%	−37.91%	−69.50%
ResNet	Entry q75	264–576	−6.29%	−22.76%	+21.44%	−17.15%
Transformer	Entry q75	552–928	−8.53%	−24.58%	−14.68%	−45.20%
MLP	Distributed q50	448–1152	−22.85%	+4.36%	−39.04%	−85.49%
ResNet	Distributed q50	264–576	−16.22%	−7.48%	−13.82%	−46.78%
Transformer	Distributed q50	552–928	−26.00%	−73.36%	−79.63%	−98.81%

Table 2: Percent changes relative to each branch's own unchanged Adam control. Negative values mean smaller loss or positive variation. MLP distributed results use the extended window; the entry screen stops at 800.

7. Interpretation and sensitivity

Entry caps improve the mean, but not every outcome. The q75 entry rule lowers average loss on all three setups, yet increases the MLP tail and ResNet maximum. It cannot be called a uniform suppression of loss peaks. **Distributed q50 improves mean loss and PV in every setup.** Its MLP tail is 4.36% worse, while ResNet and Transformer tails improve. Figure 1 shows entry caps; Figure 2 shows distributed caps, including raw-gradient and actual representation diagnostics.

Model	Additional cap variant	Mean	Tail	PV
MLP	Entry q50	−20.94%	+13.56%	−82.03%
MLP	Distributed q75	−8.44%	+35.32%	−71.44%
ResNet	Distributed q75	−17.42%	−8.31%	−17.71%

Table 3: Scope/quantile checks using the full respective windows (MLP 448–1152; ResNet 264–576). MLP entry q50 and distributed q50 share a matched worker; ResNet entry q75 and distributed q75 are also matched.

The matched-q50 MLP comparison favors distribution in mean loss (−22.85% versus −20.94%); the matched-q75 ResNet comparison does too (−17.42% versus −6.29%). These are joint changes in target scope and the resulting applied updates, not equal-norm or equal-cost comparisons. The Transformer entry and distributed experiments also differ in quantile and GPU architecture (RTX 4090 versus RTX 5090). Their control trajectories differ; we compare each cap only with its own control and do not claim an isolated Transformer depth effect. The setup comparisons are not an architecture-only ablation.

Stabilization can retain destructive geometry. Under distributed q50, the fraction of negative output-change cosines rises from 36.1% to 66.5% on ResNet and from 36.6% to 97.9% on Transformer. ResNet's mean output RMS rises from 0.567 to 0.723 while its maximum falls from 2.785 to 1.592. Transformer maximum output RMS falls from 15.365 to 1.834. Thus reduced excursions do not require forward-only motion or smaller movement at every step.

Held-out loss limits the claim. At the terminal state, distributed q50 raises test loss by 2.87% (MLP seed 0), 8.18% (ResNet), and 100.17% (Transformer). CIFAR-10 evaluation covers all 10,000 test images; WikiText-2 covers all 245,550 supervised test tokens, with token weighting. These are endpoint comparisons and may reflect different oscillation phases; no held-out trajectory or generalization improvement is established. MLP replication is reported on the last page.

Online use remains future work. These experiments use fixed data probes. Reusing each training batch's inputs could reduce capture overhead for the local cap, but refreshing data and recalibrating its thresholds require validation. Consecutive-change tracing additionally needs matching inputs across compared states: directly comparing different batches mixes parameter-induced movement with data differences. An online estimator must preserve that correspondence or explicitly account for it; neither approach is tested here.

Evidence and reproduction. Tracing motivation: Analysis 063 technical report. Cap evidence: H048 Runs 269–274 and 282–283, with exact native continuation qualification, hash-verified retrieval and retained-state audits. This report uses only existing verified cap/control records; no new training. Within Analysis 065, run `34_local_cap_report_assets.py`, then `35_build_local_cap_report.py`. Source hashes, table reductions, plotted values and PDF review are recorded in the accompanying report verification files.

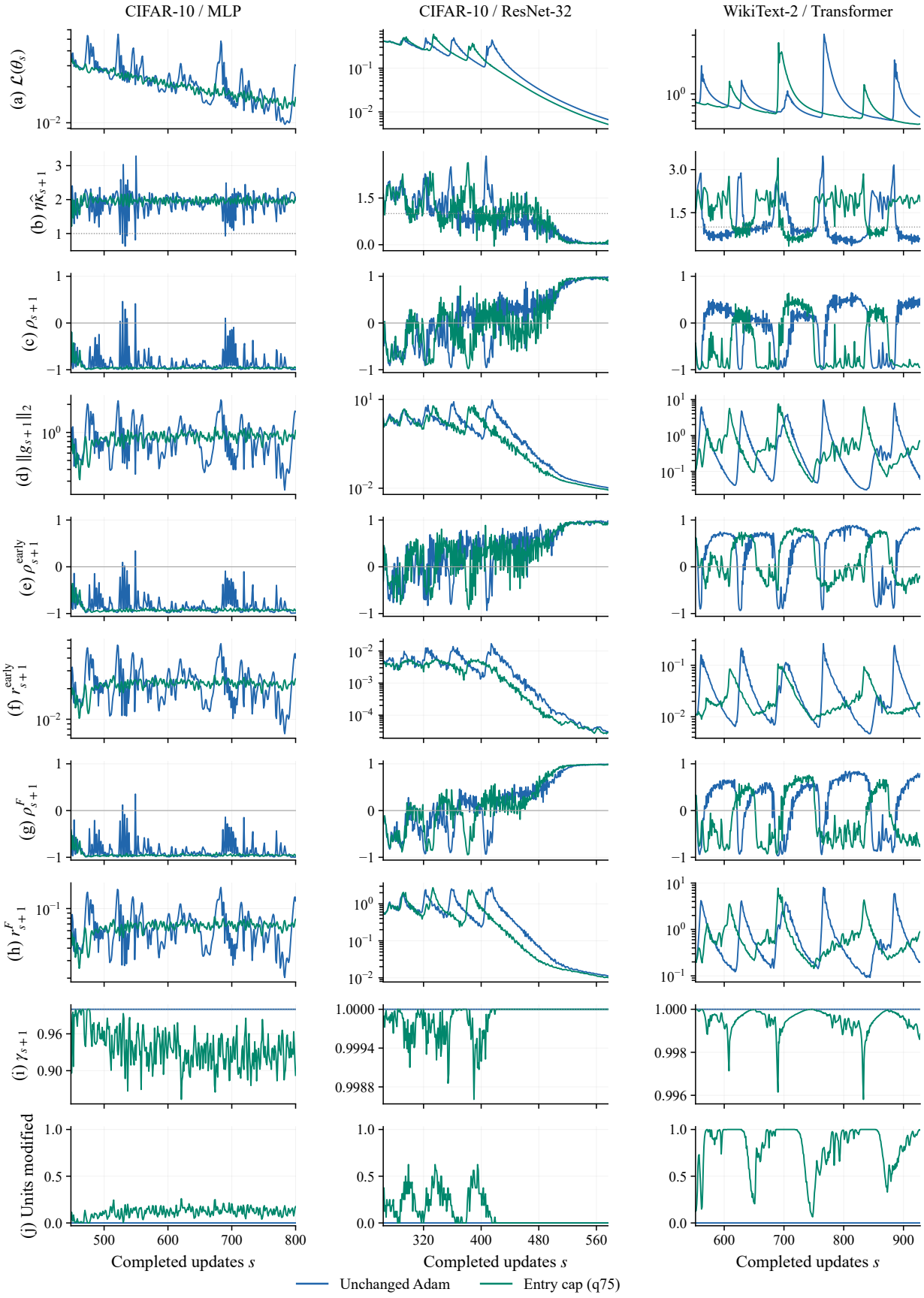


Figure 1: Entry local displacement caps (q75). Each column compares only the cap with unchanged Adam on its own worker. Loss, gradient norm and displacement RMS use logarithmic axes; $\eta\widehat{\kappa}_{s+1}$ uses a linear axis. Magnitude scales are independent by column. Early representation means first MLP tanh, ResNet conv 1 output, or first Transformer attention output. ρ uses consecutive raw gradients; ρ^{early} and ρ^{F} use consecutive outgoing representation changes. Row (i) shows the whole-parameter norm ratio $\gamma_t = \|\Delta\theta_t^{\text{cap}}\|_2 \,/\, \|\Delta\theta_t^{\text{Adam}}\|_2$ at $t = s+1$ (undefined for a zero proposal). Row (j) averages fractions of modified units across targeted modules. All curves are unsmoothed; zero-denominator cosines are gaps.

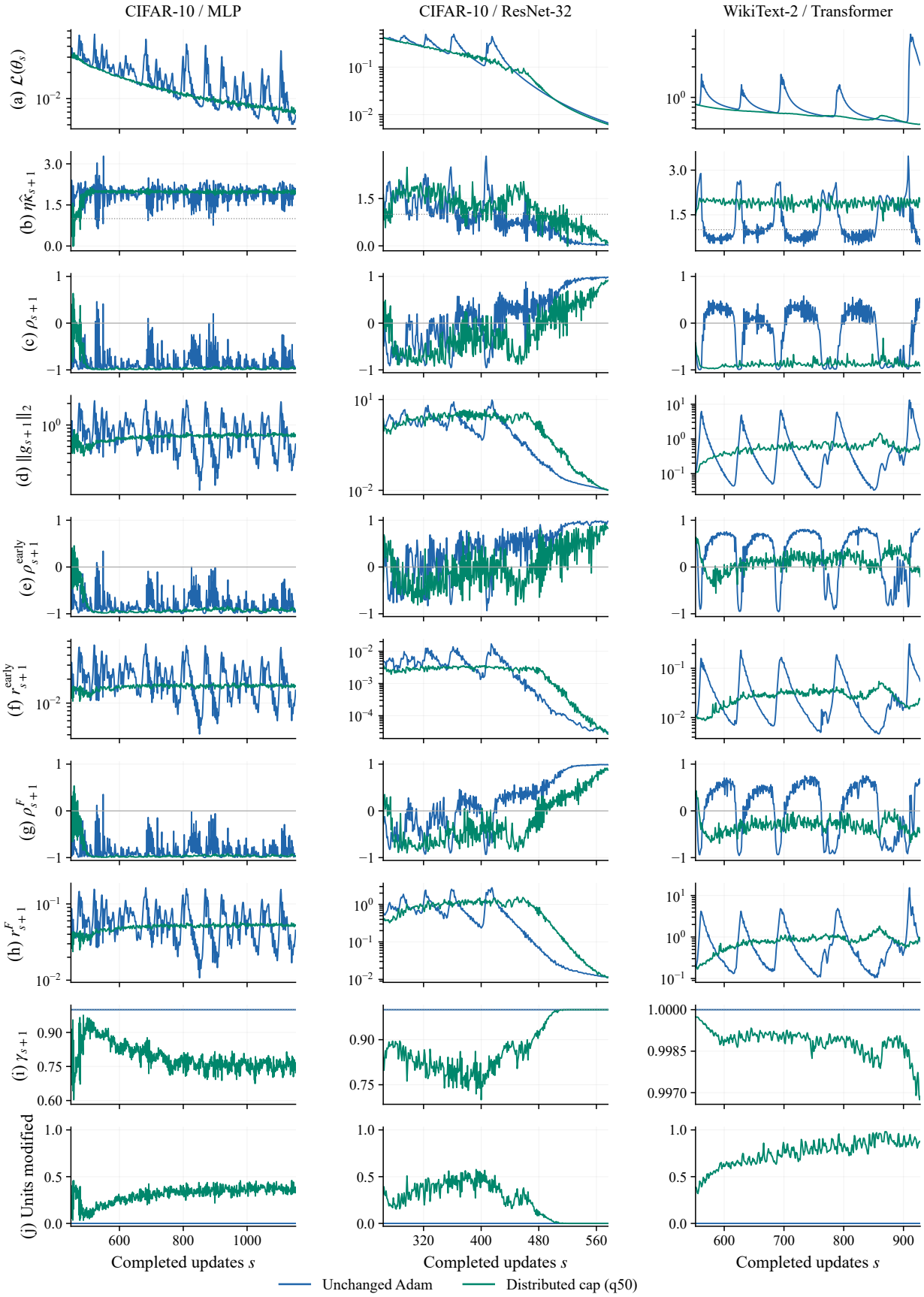


Figure 2: Distributed local displacement caps (q50). Same diagnostics and independent column scales as Figure 1; MLP is extended through 1152. Loss excursions and mean loss decline while output reversals persist. The Transformer cap targets only 402,000 of 13,828,478 parameters (2.91%), so the whole-update norm ratio stays close to one; it is not the fraction of targeted units modified. This controller bounds direct local contributions, not complete-network output displacement. Transformer baselines in this figure and Figure 1 come from different GPU architectures and must not be treated as the same numerical trajectory.

8. Replication and practical limits

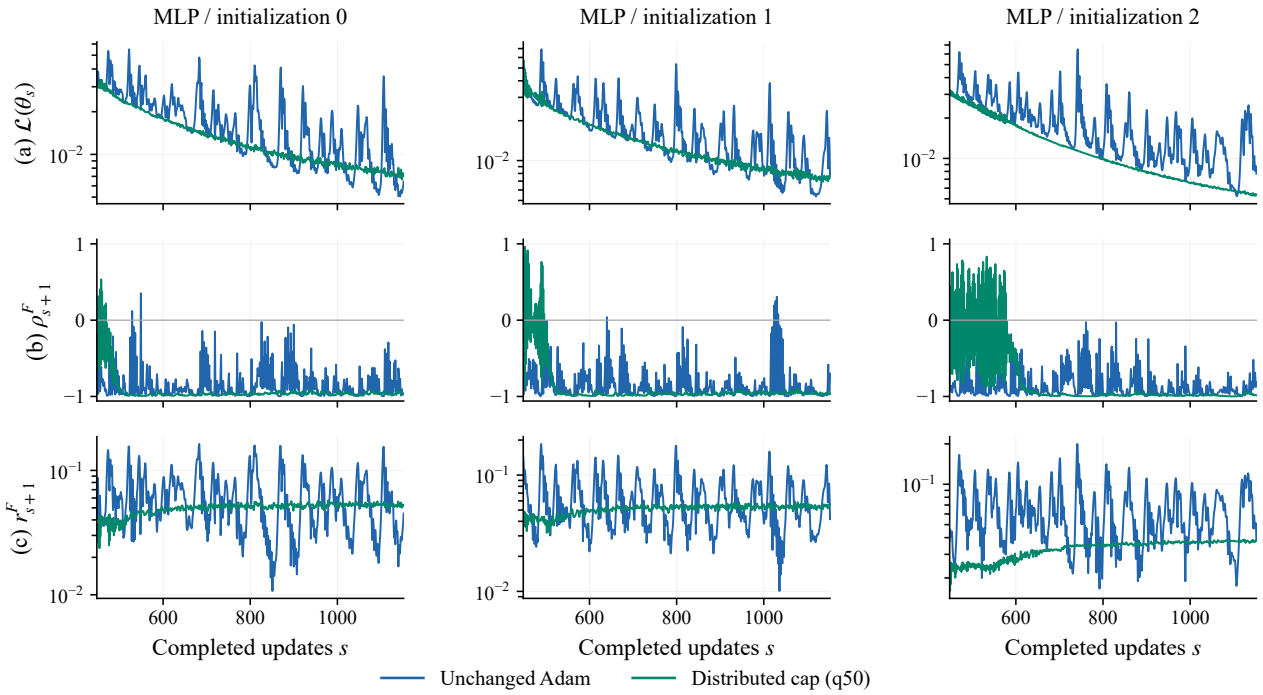


Figure 3: MLP distributed-q50 replication, states 448–1152. Initialization 0 selected the candidate; initializations 1/2 received the same rule without retuning. Loss and output RMS use logarithmic axes with independent scales per column. Each initialization has its own unchanged Adam control.

Initialization	Mean training loss	Tail loss	PV	Terminal test loss
0	−22.85%	+4.36%	−85.49%	+2.87%
1	−19.43%	−28.52%	−73.92%	+2.76%
2	−32.08%	−64.49%	−82.56%	+6.13%

Table 4: Percent changes under distributed q50. Better average training loss replicates; better terminal test loss does not.

What is bounded. The preactivation matrix $Z_t = H_t W_t^\top + \mathbf{1}_n b_t^\top$, with incoming change $\Delta H_t = H_t - H_{t-1}$, has actual change $\Delta Z_t = Z_t - Z_{t-1}$:

$$\Delta Z_t = \underbrace{H_{t-1}(\Delta W_t^{\text{cap}})^\top + \mathbf{1}_n(\Delta b_t^{\text{cap}})^\top}_{\text{capped direct contribution}} + \underbrace{\Delta H_t W_{t-1}^\top + \Delta H_t(\Delta W_t^{\text{cap}})^\top}_{\text{incoming movement and interaction}}. \tag{7}$$

Only the first term is constrained locally. Nonlinearities, normalization and attention further transform it. For the first MLP layer, the input is fixed, so $\Delta H_t = 0$. These caps preserve each unit's direct displacement direction while changing its magnitude; differing factors can change the combined direction.

Cost and scope. The implementation captures probe inputs, evaluates direct affine products in FP64 and checks final FP32 rounding. It requires no Jacobian or SVD. Instrumented control/cap seconds per update are 0.01376/0.05797 (MLP), 3.53170/3.83282 (ResNet), and 7.80567/7.95608 (Transformer). Concurrent Spark work makes the MLP timings non-isolated; these include diagnostics, audit writes and terminal evaluation, not production optimizer overhead. Fixed prefix probes and preservation of ResNet's native normalization group are essential; future padded probes need valid-position masking in local scores too.

Conclusion. Selective representation-amplitude control is a promising stabilization intervention in these deterministic full-objective continuations. Three MLP initializations and one per larger model support a limited transfer result. Selected starting states, local calibration, unequal windows and the absence of stochastic-minibatch tests limit generality. The next validation should compare additional starting states/seeds and repeated held-out measurements at matched training progress.